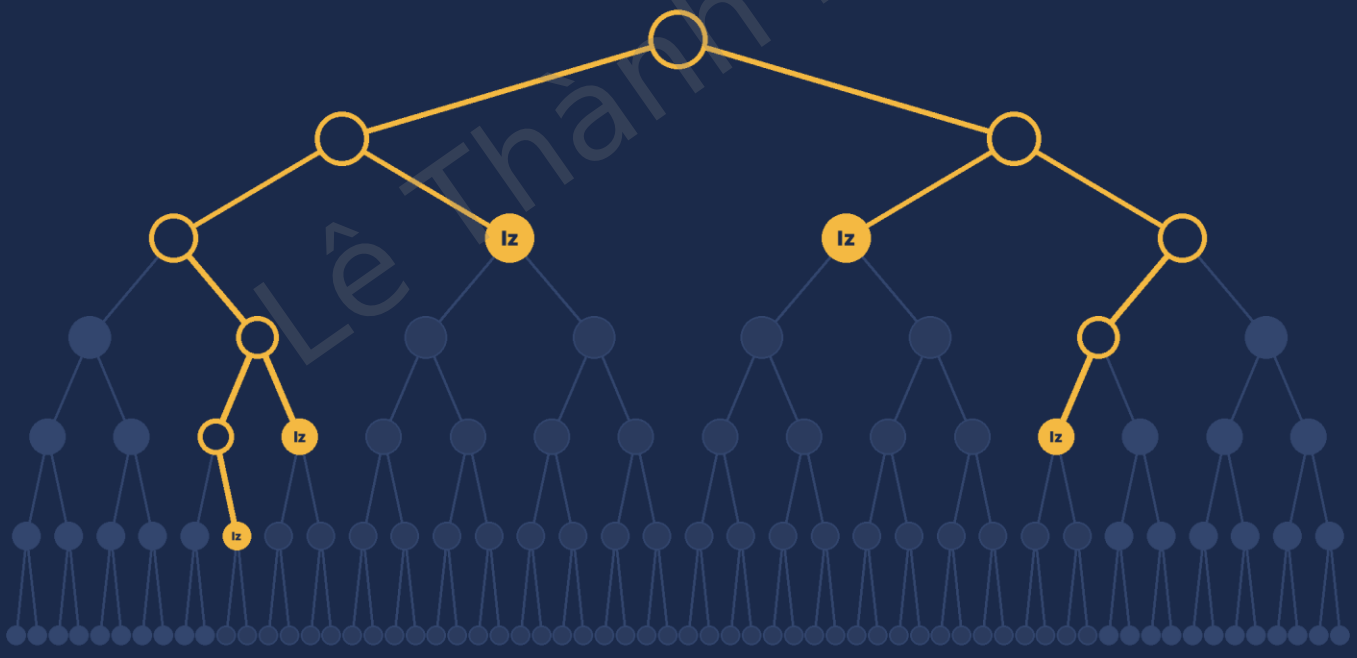
LAZY
VÀ TRUY VẤN
OFFLINE · ONLINE



Mảng hiệu · Cây phân đoạn · Fenwick · Lazy · Mo · DSU · Bền vững · Chia căn

sum[id] = sum[2id] + sum[2id + 1]
mỗi nút giữ thông tin một đoạn

update(11, 52)
64 lá, chỉ dừng ở 5 nút



Lê Thành Nam

a[l..r] += v → O(log n) nút + nhãn lazy

01	02	03	04	05
Truy vấn đoạn	Kĩ thuật lazy	Lazy nâng cao	Truy vấn offline	Online, kết hợp

TẬP 1 · LÝ THUYẾT
15 bài học · 5 chương · lý thuyết đầy đủ trước, bài toán sau

Fenwick · Mo · DSU · Chặt nhị phân song song · Bền vững

Tổng hợp và biên soạn: Lê Thành Nam

LỜI NÓI ĐẦU

"Cho một dãy số, hãy trả lời thật nhiều câu hỏi về các đoạn con của nó" là dạng bài xuất hiện trong hầu hết các đề thi học sinh giỏi Tin học, từ cấp tỉnh tới quốc gia và quốc tế. Tổng của một đoạn, giá trị lớn nhất của một đoạn, số giá trị khác nhau trong một đoạn, phần tử nhỏ thứ k của một đoạn; xen giữa các câu hỏi là các lệnh sửa: sửa một phần tử, cộng thêm vào cả một đoạn, gán cả một đoạn bằng một giá trị. Với n và số câu hỏi q cỡ $2 \cdot 10^5$, cách làm "duyệt cả đoạn cho mỗi câu hỏi" cần khoảng 10^{10} phép tính, tức vài phút. Đề bài chỉ cho một giây.

Cuốn sách này dạy hai nhóm kĩ thuật để vượt qua giới hạn đó.

- Nhóm thứ nhất là các **cấu trúc dữ liệu trên đoạn**: mảng cộng dồn, mảng hiệu, cây phân đoạn, cây Fenwick, và đặc biệt là **kĩ thuật lazy** (lazy propagation, "lan truyền lười"): khi phải cộng thêm v vào cả một đoạn, ta không sửa từng phần tử mà chỉ ghi lại một món nợ ở O(log n) nút, rồi chỉ trả nợ khi thật sự cần.
- Nhóm thứ hai là các **cách tổ chức câu hỏi**. Nếu được biết trước mọi câu hỏi (**offline**), ta được phép sắp xếp lại chúng, gom chúng lại, thậm chí đi ngược thời gian; nhiều bài trở nên dễ hẳn. Nếu đề bài bắt buộc trả lời từng câu ngay (**online**), ta cần những cấu trúc mạnh hơn: cây phân đoạn bền vững, chia căn.

Bộ tài liệu gồm ba tập đi cùng nhau:

- Tập 1: Lý thuyết (cuốn sách này) gồm 15 bài học chia thành 5 chương: Truy vấn đoạn cơ bản (mảng cộng dồn, mảng hiệu, cây phân đoạn, cây Fenwick); Kĩ thuật lazy (cộng đoạn, gán đoạn, hợp các nhãn, nút mang nhiều thông tin); Lazy nâng cao (rời rạc hoá, cây động, đường quét, khấu hao); Truy vấn offline (sắp xếp truy vấn, thuật toán Mo, đi ngược thời gian với DSU); Truy vấn online và kĩ thuật kết hợp (chặt nhị phân song song, cây bền vững, chia căn).
- Tập 2: Bài tập gồm 48 bài: mỗi bài học có ba bài tập ở ba mức độ, Bài 12 có thêm ba bài nâng cao về đi ngược thời gian; có ràng buộc theo subtask như đề thi thật.
- Tập 3: Hướng dẫn giải phân tích từng bài tập từ cách ngây thơ đến cách tối ưu, kèm chương trình hoàn chỉnh và bảng điểm thực tế của từng cách làm, kể cả những cách sai hoặc đúng nhưng chậm.

Sách được viết cho người mới bắt đầu với cấu trúc dữ liệu: chỉ cần biết mảng, vòng lặp, hàm đệ quy và sắp xếp. Mỗi bài học trình bày đầy đủ lý thuyết trước (định nghĩa, định lí, chứng minh, ví dụ chạy tay, hình vẽ), rồi mới vào phần bài toán để cài đặt. Mọi con số trong sách (thời gian chạy, kết quả chạy tay) đều là số đo và kết quả thật trên máy. Mọi chương trình đều đã được đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên.

Một điều đặc biệt của chủ đề này: **chương trình sai thường vẫn chạy rất nhanh và in ra những con số trông rất hợp lí**. Quên đẩy nhãn xuống trước khi đi vào con, hợp hai nhãn sai thứ tự, dùng 0 làm dấu "không có nhãn" trong khi 0 cũng là một giá trị hợp lệ, nén toạ độ làm mất khe hở giữa hai điểm: những lỗi ấy chỉ lộ ra ở vài test được thiết kế riêng. Vì vậy sách luôn giải thích vì sao từng dòng lệnh đúng, và Tập 3 giữ lại những lời giải sai để em thấy chúng mất bao nhiêu điểm.

Lê Thành Nam

Chương trình được viết bằng C++ theo chuẩn C++14, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học gồm hai phần lớn:

Phần	Nội dung
Mục tiêu, Khởi động	Những gì em sẽ làm được sau bài học, và một tình huống dẫn tới ý tưởng chính.
PHẦN LÝ THUYẾT	Định nghĩa, định lí, chứng minh, ví dụ chạy tay, hình minh họa. Đọc phần này với giấy bút, vẽ lại cây phân đoạn bằng tay.
PHẦN BÀI TOÁN	Một bài toán lập trình dùng đúng lý thuyết vừa học: cách chậm (kèm số đo thời gian thật), cách nhanh, chương trình hoàn chỉnh, phân tích độ phức tạp.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Luyện tập	Liên hệ tới các bài tập ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
ĐỊNH NGHĨA	Khái niệm mới, cần hiểu chính xác.
ĐỊNH LÍ	Kết quả quan trọng, luôn kèm chứng minh.
CHỨNG MINH	Lập luận vì sao định lí đúng.
VÍ DỤ	Chạy tay trên số cụ thể.
CÔNG THỨC	Công thức cần nhớ.
Ý TƯỞNG	Hướng giải hoặc các bước của một phương pháp.
BÀI TOÁN	Đề bài lập trình của bài học.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp và số đo thời gian thật.
VÌ SAO ĐÚNG?	Giải thích thêm một bước khó.
LƯU Ý	Chi tiết dễ bỏ sót.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Kí hiệu. Mảng được đánh số từ 1: a[1], a[2], ..., a[n]; đoạn [l, r] gồm các vị trí l, l + 1, ..., r. Cây phân đoạn được lưu trong mảng, nút gốc có số hiệu 1, nút id có hai con 2·id và 2·id + 1; "nút id quản lí đoạn [l, r]" nghĩa là nút id lưu thông tin của a[l..r]. log n luôn hiểu là $\log_2 n$; với $n = 2 \cdot 10^5$ thì $\log n \approx 18$. Trong hình vẽ, ô đỏ cam thường là đối tượng đang xét, ô xanh lá là kết quả, ô vàng là phần cần chú ý, ô xám là phần bị bỏ qua.

Thời gian chạy. Các số đo trong sách là thời gian CPU trên một máy tính cá nhân thông thường, biên dịch với `g++ -O2 -std=c++14`. Máy chấm của em có thể nhanh hơn hoặc chậm hơn vài lần, nhưng **tỉ lệ** giữa các cách làm thì không đổi: một thuật toán O(n·q) chậm hơn thuật toán O((n + q) log n) hàng nghìn lần ở bất kì máy nào.

Lê Thành Nam

Lời khuyên khi học. Với mỗi cấu trúc dữ liệu, hãy tự vẽ cây với n = 8 và chạy tay một lệnh sửa, một câu hỏi. Với mỗi chương trình, hãy tự hỏi: thông tin ở nút này có còn đúng không sau thao tác vừa rồi? Nhãn ở nút này có nghĩa là gì? Chỉ khi tự viết lại được chương trình mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH HỌC VỚI TÀI LIỆU NÀY **4**

CHƯƠNG 1. TRUY VẤN ĐOẠN CƠ BẢN **7**

Bài 1. Mảng cộng dồn và mảng hiệu 7
Bài 2. Cây phân đoạn: sửa một điểm, hỏi một đoạn 15
Bài 3. Cây Fenwick 22

CHƯƠNG 2. KĨ THUẬT LAZY **29**

Bài 4. Lazy propagation: cập nhật cả đoạn 29
Bài 5. Hợp các nhãn lazy 37
Bài 6. Nút mang nhiều thông tin 44

CHƯƠNG 3. LAZY NÂNG CAO **50**

Bài 7. Rời rạc hoá và cây phân đoạn động 50
Bài 8. Đường quét và cây phân đoạn đếm phủ 57
Bài 9. Tìm kiếm trên cây và cập nhật khấu hao 63

CHƯƠNG 4. TRUY VẤN OFFLINE **70**

Bài 10. Sắp xếp lại truy vấn 70
Bài 11. Thuật toán Mo 77
Bài 12. Đi ngược thời gian với DSU 83

CHƯƠNG 5. TRUY VẤN ONLINE VÀ KĨ THUẬT KẾT HỢP **91**

Bài 13. Chặt nhị phân song song 91
Bài 14. Cây phân đoạn bền vững 97
Bài 15. Chia căn: lazy trên từng khối 103

PHỤ LỤC: BẢNG TRA CỨU NHANH **109**

Lê Thành Nam

CHƯƠNG 1

TRUY VẤN ĐOẠN CƠ BẢN

Mọi kĩ thuật trong cuốn sách này đều trả lời cùng một loại câu hỏi: ***thông tin của một đoạn con*** *a[l..r] là gì, trong khi dãy a liên tục bị sửa. Chương này xây ba công cụ nền tảng:* ***mảng cộng dồn và mảng hiệu*** *(trả lời trong O(1) khi dãy không đổi hoặc khi mọi lệnh sửa đến trước mọi câu hỏi),* ***cây phân đoạn*** *(mỗi lệnh sửa một phần tử và mỗi câu hỏi trên đoạn đều chỉ mất O(log n)) và* ***cây Fenwick*** *(ngắn gọn, nhanh, đủ dùng cho phép cộng). Chương 2 sẽ nâng cây phân đoạn lên để sửa cả một đoạn trong O(log n) bằng kĩ thuật lazy.*

■ TRONG CHƯƠNG NÀY

- **Bài 1.** Mảng cộng dồn và mảng hiệu
- **Bài 2.** Cây phân đoạn: sửa một điểm, hỏi một đoạn
- **Bài 3.** Cây Fenwick

BÀI 1 Mảng cộng dồn và mảng hiệu

Học xong bài này, em sẽ:

✓ Phân biệt **truy vấn** (hỏi) và **cập nhật** (sửa) trên đoạn; biết khi nào dãy là tĩnh, khi nào là động.

✓ Tính tổng đoạn a[l..r] trong O(1) bằng **mảng cộng dồn**.

✓ Cộng v vào cả đoạn a[l..r] trong O(1) bằng **mảng hiệu**, rồi khôi phục dãy trong O(n).

✓ Mở rộng hai kĩ thuật trên sang **bảng hai chiều**.

■ KHỞI ĐỘNG

Một khu vườn có $n = 10^6$ cái cây xếp thành hàng. Người làm vườn tưới $m = 10^6$ lần, lần thứ j tưới mọi cây từ l_j tới r_j, làm mỗi cây cao thêm v_j xăng-ti-mét. Hỏi cuối cùng mỗi cây cao bao nhiêu? Chương trình "tưới từng cây" cần tới 10^{12} phép cộng trong trường hợp xấu nhất. Vậy mà chỉ cần **hai phép cộng cho mỗi lần tưới** và một lần duyệt cuối cùng, ta có đáp số trong một phần tư giây.

PHẦN LÝ THUYẾT

1. Bài toán truy vấn đoạn

Cho dãy a[1..n]. Một **truy vấn** (câu hỏi) hỏi về một đoạn con a[l..r]: tổng, giá trị nhỏ nhất, giá trị lớn nhất, ... Một **cập nhật** (lệnh sửa) làm thay đổi dãy: sửa một phần tử, hoặc sửa cả một đoạn.

■ TĨNH, ĐỘNG, OFFLINE, ONLINE

- Dãy **tĩnh**: không có lệnh sửa, chỉ có câu hỏi. Dãy **động**: lệnh sửa và câu hỏi xen kẽ nhau.
- Bài toán **offline**: chương trình được đọc hết mọi lệnh trước rồi mới phải in đáp số, nên có thể xử lí các lệnh theo thứ tự tuỳ ý.
- Bài toán **online**: phải trả lời câu hỏi thứ i trước khi biết câu hỏi thứ i + 1 (thường vì đề bài mã hoá câu hỏi sau bằng đáp số của câu trước).

Cách làm ngây thơ cho mọi bài toán truy vấn đoạn là duyệt cả đoạn: $O(r - l + 1)$ cho mỗi lệnh, tức $O(n)$ trong trường hợp xấu nhất. Với n và q cùng cỡ $2 \cdot 10^5$, tổng cộng là $4 \cdot 10^{10}$ phép tính. Bài học này giải quyết trường hợp đơn giản nhất: phép toán là **phép cộng**, dãy tĩnh hoặc "sửa xong rồi mới hỏi".

2. Mảng cộng dồn

■ MẢNG CỘNG DỒN

Mảng cộng dồn (prefix sum) của dãy a[1..n] là dãy s[0..n] với s[0] = 0 và

$$s[i] = a[1] + a[2] + \ldots + a[i] = s[i - 1] + a[i] \quad (1 \le i \le n).$$

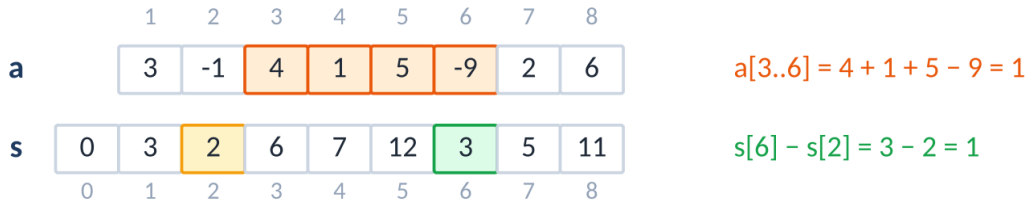
■ TỔNG MỘT ĐOẠN

Với mọi $1 \le l \le r \le n$: $a[l] + a[l + 1] + \ldots + a[r] = s[r] - s[l - 1]$.

Lê Thành Nam

■ CHỨNG MINH

s[r] = (a[1] + ... + a[l − 1]) + (a[l] + ... + a[r]) = s[l − 1] + (a[l] + ... + a[r]). Chuyển s[l − 1] sang vế trái. Nhờ quy ước s[0] = 0, công thức vẫn đúng khi l = 1.



Hình 1.1. *Tổng a[3..6] bằng hiệu hai giá trị của mảng cộng dồn*

■ VÍ DỤ

a = (3, −1, 4, 1, 5, −9, 2, 6). Mảng cộng dồn s = (0, 3, 2, 6, 7, 12, 3, 5, 11). Tổng a[3..6] = 4 + 1 + 5 − 9 = 1, và đúng là s[6] − s[2] = 3 − 2 = 1. Tổng cả dãy là s[8] = 11.

Dựng mảng s mất O(n), sau đó **mỗi câu hỏi chỉ mất O(1)**: hai lần đọc mảng và một phép trừ. Đó là cách nhanh nhất có thể cho dãy tĩnh.

■ KHI NÀO KHÔNG DÙNG ĐƯỢC?

- Phép toán phải **có phép ngược**: tổng dùng được vì có phép trừ; XOR cũng dùng được (phép ngược của XOR là chính nó). Nhưng **min, max thì không**: biết min(a[1..r]) và min(a[1..l − 1]) không suy ra được min(a[l..r]).
- Dãy phải **không đổi**: sửa một phần tử a[i] làm thay đổi s[i], s[i + 1], ..., s[n], mất O(n). Bài 2 và Bài 3 khắc phục điều này.

Lê Thành Nam

3. Mảng hiệu

Bây giờ xét bài toán ngược lại: có rất nhiều lệnh "cộng v vào mọi phần tử của đoạn [l, r]", và chỉ cần biết dãy **sau khi** thực hiện hết các lệnh đó.

■ MẢNG HIỆU

Mảng hiệu (difference array) của dãy a[1..n] là dãy d[1..n + 1] với quy ước a[0] = 0, a[n + 1] = 0 và

$$d[i] = a[i] - a[i-1] \quad (1 \le i \le n+1).$$

Mảng hiệu là "phép ngược" của mảng cộng dồn: cộng dồn d ta lại được a, vì d[1] + d[2] + ... + d[i] = (a[1] − a[0]) + (a[2] − a[1]) + ... + (a[i] − a[i − 1]) = a[i] (tổng rút gọn, các số hạng giữa triệt tiêu nhau).

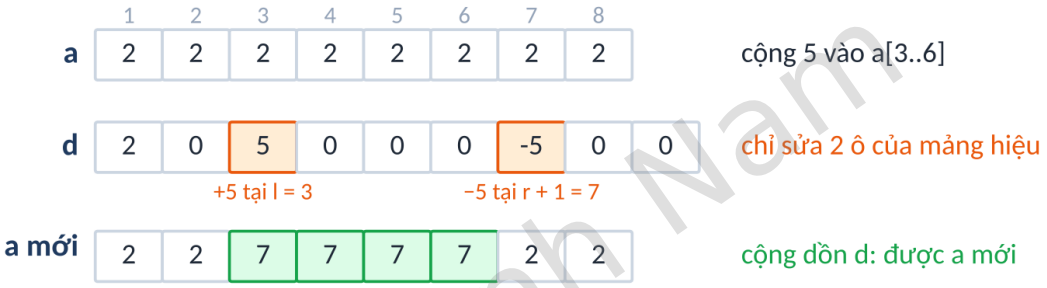
■ CỘNG MỘT ĐOẠN = SỬA HAI Ô

Cộng v vào a[l], a[l + 1], ..., a[r] tương đương với hai thao tác trên mảng hiệu: **d[l] += v** và **d[r + 1] −= v**. Mọi ô khác của d không đổi.

■ CHỨNG MINH

Xét d[i] = a[i] − a[i − 1] sau khi cộng:

- Nếu l < i ≤ r: cả a[i] và a[i − 1] cùng tăng v, hiệu không đổi.
- Nếu i = l: a[l] tăng v còn a[l − 1] không đổi, nên d[l] tăng v.
- Nếu i = r + 1: a[r + 1] không đổi còn a[r] tăng v, nên d[r + 1] giảm v.
- Các vị trí còn lại: cả hai số đều không đổi.



Hình 1.2. Cộng 5 vào a[3..6]: chỉ hai ô của mảng hiệu thay đổi

Vậy mỗi lệnh cộng đoạn chỉ mất O(1). Sau khi thực hiện hết m lệnh, cộng dồn d một lần (O(n)) là được dãy cuối cùng. Tổng cộng **O(n + m)** thay vì O(n·m).

■ VÍ DỤ

a = (1, 2, 3, 4, 5), mảng hiệu d = (1, 1, 1, 1, 1, −5). Ba lệnh: cộng 2 vào [1, 3], cộng 1 vào [2, 5], cộng 10 vào [4, 4].

- Sau lệnh 1: d[1] += 2, d[4] −= 2, d = (3, 1, 1, −1, 1, −5).
- Sau lệnh 2: d[2] += 1, d[6] −= 1, d = (3, 2, 1, −1, 1, −6).
- Sau lệnh 3: d[4] += 10, d[5] −= 10, d = (3, 2, 1, 9, −9, −6).

Cộng dồn d: 3, 5, 6, 15, 6. Kiểm tra trực tiếp: (1 + 2, 2 + 2 + 1, 3 + 2 + 1, 4 + 1 + 10, 5 + 1) = (3, 5, 6, 15, 6).

4. "Sửa trước, hỏi sau": lần gặp đầu tiên với offline

Mảng hiệu chỉ cho biết dãy sau khi **đã thực hiện xong mọi lệnh cộng**. Nếu câu hỏi xen giữa các lệnh cộng ("cộng vào [2, 5]; hỏi a[3]; cộng vào [1, 4]; hỏi tổng [2, 3]; ..."), ta phải cộng dồn lại sau mỗi lệnh, mất O(n) mỗi lần. Bảng sau tóm tắt chi phí:

Tình huống	Lệnh cộng đoạn	Câu hỏi	Công cụ

Lê Thành Nam

Tình huống	Lệnh cộng đoạn	Câu hỏi	Công cụ
Dãy tĩnh, hỏi tổng đoạn	không có	O(1)	mảng cộng dồn
Mọi lệnh cộng đến trước mọi câu hỏi	O(1)	O(1) (sau O(n) dựng lại)	mảng hiệu rồi cộng dồn
Lệnh cộng và câu hỏi xen kẽ	O(log n)	O(log n)	cây Fenwick, cây phân đoạn (Bài 3, Bài 4)

Hàng thứ hai là một bài toán **offline**: nếu đề bài cho phép đọc hết dữ liệu trước, ta có thể tự sắp xếp lại "mọi lệnh sửa trước, mọi câu hỏi sau" khi điều đó không làm thay đổi đáp số. Chương 4 sẽ khai thác ý tưởng này triệt để hơn nhiều.

5. Hai chiều

Với bảng a[1..n][1..m], mảng cộng dồn hai chiều là S[i][j] = tổng mọi a[x][y] với x ≤ i, y ≤ j, tính bằng

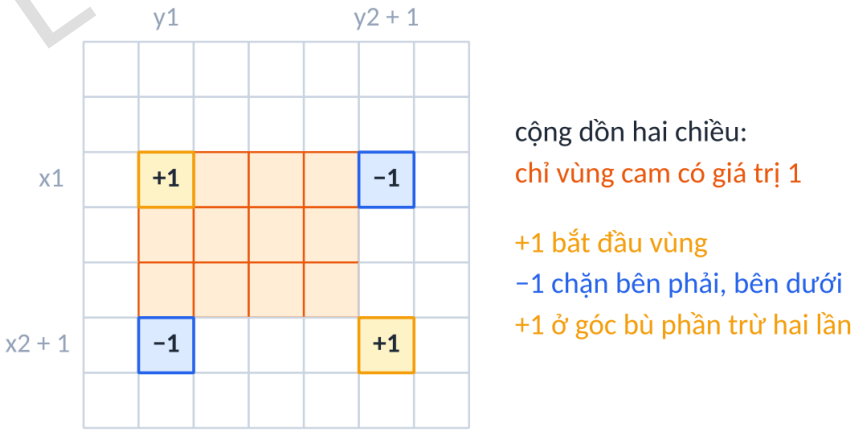
$$S[i][j] = a[i][j] + S[i-1][j] + S[i][j-1] - S[i-1][j-1].$$

Phần S[i − 1][j − 1] bị cộng hai lần (trong S[i − 1][j] và trong S[i][j − 1]) nên phải trừ đi một lần: đó là **bao hàm loại trừ**. Tổng hình chữ nhật (x1, y1) – (x2, y2) cũng theo cùng lập luận:

■ CÔNG THỨC

$$\text{Tổng}(x1..x2, y1..y2) = S[x2][y2] - S[x1-1][y2] - S[x2][y1-1] + S[x1-1][y1-1].$$

Ngược lại, "cộng v vào mọi ô của hình chữ nhật (x1, y1) – (x2, y2)" trên mảng hiệu hai chiều là bốn thao tác ở bốn góc: d[x1][y1] += v, d[x1][y2 + 1] −= v, d[x2 + 1][y1] −= v, d[x2 + 1][y2 + 1] += v. Cộng dồn hai chiều (theo công thức S ở trên) ta được bảng sau khi cộng.



Hình 1.3. *Mảng hiệu hai chiều: bốn góc của hình chữ nhật*

Lê Thanh Nam

■ GHI NHỚ

- Dãy tĩnh, phép toán có phép ngược (cộng, XOR): **mảng cộng dồn**, câu hỏi O(1).
- Nhiều lệnh cộng đoạn, chỉ cần dãy cuối cùng: **mảng hiệu**, d[l] += v, d[r + 1] −= v, rồi cộng dồn.
- Hai chiều: bao hàm loại trừ với bốn góc.
- Min, max không có phép ngược; dãy bị sửa xen kẽ với câu hỏi: cần cấu trúc khác (Bài 2, 3, 4).

PHẦN BÀI TOÁN

■ BÀI TOÁN: TUOICAY

Hàng cây có n cây, cây thứ i cao h_i. Người làm vườn tưới m lần; lần thứ j tưới mọi cây từ l_j tới r_j và làm mỗi cây đó cao thêm v_j. Hãy in chiều cao cuối cùng của từng cây.

Dữ liệu vào: Dòng đầu gồm n, m ($1 \le n, m \le 10^6$). Dòng thứ hai gồm $h_1, ..., h_n$ ($0 \le h_i \le 10^6$). Mỗi dòng trong m dòng tiếp theo gồm l, r, v ($1 \le l \le r \le n$; $1 \le v \le 10^6$).

Kết quả: n số là chiều cao cuối cùng của các cây.

Dữ liệu vào	Kết quả	Giải thích
5 3 1 2 3 4 5 1 3 2 2 5 1 4 4 10	3 5 6 15 6	Ví dụ ở mục 3.
3 2 0 0 0 1 3 7 2 2 1	7 8 7	Cây giữa được tưới hai lần.

Lê Thanh Nam

6. Cách chậm: tưới từng cây

Làm đúng như đề bài: với mỗi lần tưới, cộng v vào từng cây trong đoạn.

▣ Tưới từng cây, O(n·m)

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    int n, m;
    cin >> n >> m;
    vector<ll> h(n + 1);
    for (int i = 1; i <= n; i++) cin >> h[i];
    for (int j = 0; j < m; j++) {
        int l, r; ll v;
        cin >> l >> r >> v;
        for (int i = l; i <= r; i++) h[i] += v;   // tưới từng cây: O(r - l + 1)
```

Tưới từng cây, O(n·m)

```
    }
    for (int i = 1; i <= n; i++) cout << h[i] << (i < n ? ' ' : '\n');
    return 0;
}
```

7. Mảng hiệu

Áp dụng mục 3: mỗi lần tưới sửa hai ô của mảng hiệu, cuối cùng cộng dồn một lần.

TUOICAY.CPP: Mảng hiệu

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("TUOICAY.INP", "r")) {
        freopen("TUOICAY.INP", "r", stdin);
        freopen("TUOICAY.OUT", "w", stdout);
    }
    int n, m;
    cin >> n >> m;
    vector<ll> h(n + 2, 0), d(n + 2, 0);
    for (int i = 1; i <= n; i++) cin >> h[i];
    for (int i = 1; i <= n; i++) d[i] = h[i] - h[i - 1];   // mảng hiệu của h
    for (int j = 0; j < m; j++) {
        int l, r; ll v;
        cin >> l >> r >> v;
        d[l] += v;          // O(1) cho mỗi lần tưới
        d[r + 1] -= v;
    }
    ll cur = 0;
    for (int i = 1; i <= n; i++) {
        cur += d[i];        // cộng dồn mảng hiệu: được chiều cao cuối cùng
        cout << cur << (i < n ? ' ' : '\n');
    }
    return 0;
}
```

Lê Thành Nam

■ ĐẾM SỐ PHÉP TÍNH

- Tưới từng cây: mỗi lần tưới trung bình duyệt khoảng n/3 cây. Với $n = m = 10^5$ mất **1,14 giây**; với $n = m = 2 \cdot 10^5$ mất **4,36 giây** (gấp đôi n và m thì thời gian gấp bốn).
- Mảng hiệu: O(n + m). Với $n = m = 10^5$ chỉ **0,026 giây**; với $n = m = 10^6$ mất **0,25 giây**, phần lớn là thời gian đọc dữ liệu.